Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquín



Ejercicio 6

Estudiantes:
Emily Lucila Menchú Menchú - 20472

Fecha: 03/11/2026

LINK GITHUB: https://github.com/emilymenchu/ejercicio-6.git

Ejercicio 6

Análisis

Requisitos:

- Representar servicios de pagos, logística y facturación FEL.
- Cargar al menos 10 integraciones al iniciar.
- Guardar todas las integraciones en una sola lista polimórfica.
- Conservar el identificador, configuración, estado de conexión y latencia de cada integración.
- Mostrar las integraciones activas y su estado de conexión.
- Buscar integraciones por identificador y consultar su configuración.
- Ordenar las integraciones por latencia usando Comparable.
- Usar interfaces para definir las operaciones que permite cada servicio.
- Ignorar las operaciones no admitidas sin interrumpir el programa.
- Permitir agregar nuevos proveedores sin cambiar la lógica central.
- Aplicar MVC para separar el modelo, las vistas y el controlador.
- Incluir un menú en una interfaz gráfica.
- Utilizar herencia, polimorfismo, encapsulación y sobrecarga donde corresponda.
- Sobrescribir toString y, cuando sea útil, equals.
- Mantener el código y el diagrama UML consistentes.

Operaciones

- Cargar las integraciones iniciales en el catálogo.
- Listar las integraciones activas y su estado de conexión.
- Buscar una integración por su identificador.
- Mostrar la configuración de una integración.
- Ordenar las integraciones por su latencia promedio.
- Consultar el estado de una operación o de un envío.
- Cotizar un envío mediante un proveedor logístico.
- Realizar cobros y reintegros mediante servicios de pago.
- Generar facturas o guías de envío mediante servicios que lo permitan.
- Consultar información de auditoría de los servicios que implementen Auditable.
- Comprobar si una integración admite la operación solicitada.
- Mostrar el resultado de la operación o indicar que no aplica.
- Permitir estas acciones desde la interfaz gráfica.

Condiciones

- El identificador de cada integración debe ser único y no estar vacío.
- El nombre y los parámetros de configuración necesarios no deben estar vacíos.
- La latencia promedio debe ser mayor o igual a cero.

- Una integración debe existir en el catálogo antes de consultar sus datos o solicitar una operación.
- Para ejecutar una operación simulada, la integración debe estar activa y conectada.
- Los montos de cobros y reintegros deben ser mayores que cero.
- Las consultas de seguimiento deben incluir un identificador de envío.
- Las cotizaciones deben incluir un origen, un destino y un peso mayor que cero.
- Para generar un documento, deben proporcionarse los datos necesarios según su tipo.
- Cada integración debe ejecutar únicamente las operaciones que admite.
- Si una operación no aplica, se debe ignorar la solicitud y continuar el proceso sin lanzar una excepción.
- Si el usuario ingresa datos inválidos, se debe mostrar un mensaje y permitir corregirlos.
- La interfaz gráfica deben aplicar las mismas validaciones.

Clases que se implementarán

Clase	Información a conservar	Responsabilidad
IntegracionAPI (abstracta)	Identificador, nombre, configuración, activa, estado de conexión y latencia promedio	Definir los datos comunes de las integraciones y compararlas por latencia.
PasarelaPago	Datos heredados y medio de pago	Simular cobros y reintegros.
ProveedorLogistico	Datos heredados	Simular cotizaciones, seguimiento y generación de guías.
CertificadorFEL	Datos heredados	Simular la generación de facturas y la consulta de su estado.
CatalogoIntegraciones	Una lista ArrayList<IntegracionAPI>	Registrar, listar, buscar y ordenar las integraciones.
MotorCheckout	Referencia al catálogo	Coordinar las operaciones según las capacidades de cada integración y manejar solicitudes que no aplican.
Controlador	Referencias al catálogo, al motor y a la vista	Recibir las acciones del usuario, solicitar su ejecución y actualizar la vista.

ModeloTablaIntegraciones	Identificador de serialización (serialVersionUID)	Definir las columnas y los tipos de datos de la tabla e impedir que se editen sus celdas.
VistaGrafica	Ventana, tabla, campos, botones y panel de resultados	Mostrar información, recibir entradas y presentar mensajes.
Main	No necesita atributos propios	Crear y conectar los componentes, cargar las integraciones iniciales e iniciar la ventana.

Interfaces que se usarán

Interfaz	Propósito	Clase que la implementa
Transaccional	Definir cobros y reintegros	PasarelaPago
Documentable	Definir la generación de documentos	ProveedorLogistico y CertificadorFEL
Auditable	Definir cómo obtener información de auditoría	Las tres clases concretas
Comparable<IntegracionAPI>	Permitir el ordenamiento por latencia	IntegracionAPI

Separación MVC

Componente	Clases	Función
Modelo	IntegracionAPI, sus clases hijas, CatalogoIntegraciones y MotorCheckout	Conservar los datos y ejecutar las reglas y operaciones.
Vista	VistaGrafica	Mostrar información y obtener las entradas del usuario.
Controlador	Controlador	Gestionar los eventos de la vista y coordinar las acciones con el modelo.

Inicio	Main	Inicia el programa

Justificación de jerarquías
IntegracionAPI será abstracta y contendrá los datos comunes. De ella heredarán PasarelaPago, ProveedorLogistico y CertificadorFEL.

Polimorfismo
Todas las integraciones se guardarán en una sola lista de tipo IntegracionAPI. Las interfaces permitirán ejecutar operaciones según las capacidades de cada servicio. Si una operación no aplica, el programa continuará sin ejecutarla.

Sobrecarga
Los constructores tendrán una versión con datos básicos y otra que permita indicar también si la integración está activa y conectada.

Sobrescritura
Se implementarán compareTo para ordenar por latencia, toString para representar los objetos y los métodos de cada interfaz según el servicio.

Flujo de una operación

1. El usuario selecciona una integración en la ventana y elige realizar un reintegro.
2. La vista solicita el identificador del cobro original y el monto que desea reintegrar.
3. El controlador recibe los datos y solicita la operación al MotorCheckout.
4. El motor busca la integración en el catálogo y verifica que exista.
5. Comprueba si implementa Transaccional. Si no lo hace, devuelve “Operación no aplicable” y termina esa solicitud sin interrumpir el programa.
6. Si admite reintegros, verifica que esté activa y conectada, y que los datos sean válidos.
7. El motor solicita el reintegro mediante la interfaz Transaccional. La pasarela ejecuta su comportamiento simulado.
8. Se obtiene la información de auditoría mediante Auditable.
9. El controlador entrega el resultado a la vista.
10. La vista muestra si el reintegro se realizó, no aplicaba o no pudo completarse.

Como se incorporan nuevos servicios sin cambiar la lógica central

Nuevo servicio	Cómo se incorpora

Otra pasarela con el mismo comportamiento	Crear un objeto PasarelaPago con sus datos de configuración y agregarlo al catálogo.
Otro proveedor de entregas con el mismo comportamiento	Crear un objeto ProveedorLogistico y agregarlo al catálogo.
Otro certificador FEL con el mismo comportamiento	Crear un objeto CertificadorFEL y agregarlo al catálogo.
Un proveedor con comportamiento diferente	Crear una clase que herede de IntegracionAPI, implementar las interfaces correspondientes y agregar su instancia al catálogo.

Atributos de modelo y controlador

#	Tipo de dato	Nombre	Visibilidad	Motivo	Clase
1	String	identificador	private	Identificar la integración.	IntegracionAPI
2	String	nombre	private	Guardar el nombre del servicio.	IntegracionAPI
3	String	configuracion	private	Conservar los parámetros de la conexión simulada.	IntegracionAPI
4	boolean	activa	private	Indicar si la integración está habilitada.	IntegracionAPI
5	boolean	conectada	private	Indicar si la conexión está disponible.	IntegracionAPI

6	double	latenciaPromedio	private	Guardar el tiempo promedio de respuesta en milisegundos.	IntegracionAPI
7	String	medioPago	private	Indicar si utiliza tarjeta, transferencia o billetera digital.	PasarelaPago
8	ArrayList<IntegracionAPI>	integraciones	private	Almacenar todas las integraciones en una sola lista polimórfica.	CatalogoIntegraciones
9	CatalogoIntegraciones	catalogo	private	Acceder a las integraciones para coordinar las operaciones.	MotorCheckout
10	CatalogoIntegraciones	catalogo	private	Solicitar búsquedas, consultas y ordenamiento.	Controlador
11	MotorCheckout	motor	private	Solicitar la ejecución de operaciones.	Controlador

12	VistaGrafica	vista	private	Obtener las entradas y actualizar la información presentada.	Controlador
	boolean	iniciado	private	Evitar que los eventos de los botones se registren varias veces.	Controlador
	ArrayList<String>	nombresCampos	private final	Guardar los nombres de los campos del formulario actual.	VistaGrafica
	ArrayList<JTextField>	camposOperacion	private final	Guardar los cuadros de texto del formulario actual.	VistaGrafica
	LONG	serialVersionUID	private static final	Identificar la versión de serialización de la clase heredada de Swing.	ModeloTablaIntegraciones

Componentes de Swing para la interfaz

#	Tipo de dato	Nombre	Visibilidad	Motivo	Clase

13	JFrame	ventana	private	Contener la interfaz gráfica.	VistaGrafica
14	JTable	tablaIntegraciones	private	Mostrar el catálogo y los estados de conexión.	VistaGrafica
15	JTextField	campoIdentificador	private	Recibir el identificador que se desea buscar.	VistaGrafica
16	JButton	botonBuscar	private	Solicitar la búsqueda de una integración.	VistaGrafica
17	JButton	botonListar	private	Solicitar que se muestren las integraciones activas.	VistaGrafica
18	JButton	botonOrdenar	private	Solicitar el ordenamiento por latencia.	VistaGrafica
19	JComboBox<String>	selectorOperacion	private	Permitir elegir la operación que se realizará.	VistaGrafica
20	JPanel	panelDatosOperacion	private	Agrupar los campos necesarios para la operación elegida.	VistaGrafica
21	JButton	botonEjecutar	private	Solicitar la ejecución de la operación seleccionada.	VistaGrafica
22	JTextArea	areaDetalles	private	Mostrar la configuración de una integración.	VistaGrafica

23	JTextArea	areaResultados	private	Mostrar resultados y mensajes de auditoría.	VistaGrafica

Métodos de la interfaces

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo	Interfaz
1	String	cobrar	public	double monto	Realizar un cobro simulado.	Transaccional
2	String	reintegrar	public	String referenciaCobro, double monto	Simular la devolución de dinero de un cobro.	Transaccional
3	String	generarDocumento	public	String datos	Generar una factura o guía simulada según el servicio.	Documentable
4	String	generarAuditoria	public	String accion, String resultado	Describir la acción realizada y su resultado para auditoría.	Auditable

Métodos de IntegracionAPI

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo
5	—	IntegracionAPI	protected	String identificador, String nombre, String configuracion, double latenciaPromedio	Inicializar los datos comunes y poner la integración como activa y conectada.
6	—	IntegracionAPI	protected	String identificador, String nombre, String configuracion, double latenciaPromedio, boolean activa, boolean conectada	Inicializar los datos comunes permitiendo indicar ambos estados.
7	String	getIdentificador	public	—	Consultar el identificador.
8	String	getNombre	public	—	Consultar el nombre del servicio.
9	String	getConfiguracion	public	—	Consultar los parámetros de configuración.
10	boolean	isActiva	public	—	Consultar si la integración está habilitada.
11	boolean	isConectada	public	—	Consultar si la conexión está disponible.
12	double	getLatenciaPromedio	public	—	Consultar la latencia en milisegundos.
13	void	setConfiguracion	public	String configuracion	Actualizar los parámetros de configuración.

14	void	setActiva	public	boolean activa	Habilitar o deshabilitar la integración.
15	void	setConectada	public	boolean conectada	Actualizar el estado de conexión.
16	void	setLatenciaPromedio	public	double latenciaPromedio	Actualizar la latencia, validando que no sea negativa.
17	String	consultarEstado	public abstract	String referencia	Definir la consulta del estado de un cobro, envío o factura.
18	int	compareTo	public	IntegracionAPI otra	Comparar las integraciones por latencia para ordenarlas de menor a mayor.
19	String	toString	public	—	Devolver una representación legible de los datos comunes.

Métodos de PasarelaPago:

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo
20	—	PasarelaPago	public	String identificador, String nombre, String configuracion, double latenciaPromedio, String medioPago	Inicializar la pasarela como activa y conectada.

21	—	PasarelaPago	public	String identificador, String nombre, String configuracion, double latenciaPromedio, String medioPago, boolean activa, boolean conectada	Inicializar la pasarela indicando sus estados.
22	String	getMedioPago	public	—	Consultar el medio de pago.
23	String	cobrar	public	double monto	Simular un cobro y devolver su referencia y resultado.
24	String	reintegrar	public	String referenciaCobro, double monto	Simular un reintegro asociado a un cobro.
25	String	consultarEstado	public	String referencia	Simular la consulta del estado de una operación de pago.
26	String	generarAuditoria	public	String accion, String resultado	Devolver información de auditoría de la operación.
27	String	toString	public	—	Mostrar los datos comunes y el medio de pago.

Métodos para ProveedorLogistico

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo

28	—	ProveedorLogistico	public	String identificador, String nombre, String configuracion, double latenciaPromedio	Inicializar el proveedor como activo y conectado.
29	—	ProveedorLogistico	public	String identificador, String nombre, String configuracion, double latenciaPromedio, boolean activa, boolean conectada	Inicializar el proveedor indicando sus estados.
30	double	cotizarEnvio	public	String origen, String destino, double peso	Calcular una cotización simulada de envío.
31	String	consultarEstado	public	String referencia	Simular el seguimiento de un envío.
32	String	generarDocumento	public	String datos	Generar una guía de envío simulada.
33	String	generarAuditoria	public	String accion, String resultado	Devolver información de auditoría de la operación logística.

Métodos para CertificadorFEL

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo
34	—	CertificadorFEL	public	String identificador, String nombre, String configuracion, double latenciaPromedio	Inicializar el certificador como activo y conectado.

35	—	CertificadorFEL	public	String identificador, String nombre, String configuracion, double latenciaPromedio, boolean activa, boolean conectada	Inicializar el certificador indicando sus estados.
36	String	consultarEstado	public	String referencia	Simular la consulta del estado de una factura.
37	String	generarDocumento	public	String datos	Generar el contenido XML de una factura simulada, sin validez fiscal.
38	String	generarAuditoria	public	String accion, String resultado	Devolver información de auditoría de la operación de facturación.

Métodos para CatalogoIntegraciones

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo
39	—	CatalogoIntegraciones	public	—	Inicializar la lista polimórfica vacía.
40	void	cargarIntegracionesIniciales	public	—	Registrar al menos diez integraciones de distinta naturaleza al iniciar.

41	boolean	registrarIntegracion	public	IntegracionAPI integracion	Agregar una integración válida y con identificador único. Devolver si se registró.
42	IntegracionAPI	buscarPorIdentificador	public	String identificador	Buscar una integración y devolver null si no existe.
43	List<IntegracionAPI>	consultarIntegraciones	public	—	Permitir consultar el catálogo sin modificar directamente su lista.
44	void	ordenarPorLatencia	public	—	Ordenar la lista de menor a mayor latencia utilizando compareTo.

Metodos para MotorCheckout

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo
45	—	MotorCheckout	public	CatalogoIntegraciones catalogo	Asociar el motor con el catálogo existente.

46	String	procesarCobro	public	String identificador, double monto	Buscar la integración, validar su capacidad y disponibilidad, y solicitar el cobro.
47	String	procesarReintegro	public	String identificador, String referenciaCobro, double monto	Validar la solicitud y realizar el reintegro si corresponde.
48	String	procesarDocumento	public	String identificador, String datos	Solicitar un documento si la integración implementa Documentable.
49	String	consultarEstado	public	String identificador, String referencia	Validar la disponibilidad y consultar el estado del cobro, envío o factura.
50	String	procesarCotizacion	public	String identificador, String origen, String destino, double peso	Validar los datos y solicitar una cotización a un proveedor logístico.
51	boolean	estaDisponible	private	IntegracionAPI integracion	Comprobar que la integración esté activa y conectada.
52	String	ejecutarCobro	private	Transaccional servicio, double monto	Ejecutar el cobro mediante el contrato Transaccional.

53	String	ejecutarReintegro	private	Transaccional servicio, String referenciaCobro, double monto	Ejecutar el reintegro mediante el contrato Transaccional.
54	String	ejecutarGeneracion	private	Documentable servicio, String datos	Generar el documento mediante el contrato Documentable.
55	String	obtenerAuditoria	private	Auditable servicio, String accion, String resultado	Obtener la descripción de auditoría mediante su interfaz.

Métodos para Controlador

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo
56	—	Controlador	public	CatalogoIntegraciones catalogo, MotorCheckout motor, VistaGrafica vista	Asociar el controlador con el modelo y la vista.
57	void	iniciar	public	—	Vincular los eventos, mostrar las integraciones activas y abrir la ventana.
58	void	registrarEventos	private	—	Asociar los botones de la vista con las acciones del controlador.

59	void	listarIntegraciones	public	—	Consultar el catálogo y enviar las integraciones activas a la tabla gráfica.
60	void	buscarIntegracion	public	—	Leer el identificador, buscar la integración y mostrar sus detalles.
61	void	ordenarIntegraciones	public	—	Solicitar el ordenamiento por latencia y actualizar la tabla.
62	void	ejecutarOperacion	public	—	Leer la operación y sus datos, llamar al método correspondiente del motor y presentar el resultado.

Métodos para la VistaGrafica

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo
63	—	VistaGrafica	public	—	Crear la ventana y sus componentes.

64	void	inicializarComponentes	private	—	Construir y distribuir la tabla, los campos, los botones y los paneles.
65	void	mostrarVentana	public	—	Hacer visible la ventana.
66	String	obtenerIdentificadorBusqueda	public	—	Obtener el identificador escrito en el buscador.
67	String	obtenerIdentificadorSeleccionado	public	—	Obtener el identificador de la fila seleccionada o null si no hay selección.
68	String	obtenerOperacionSeleccionada	public	—	Consultar la operación elegida.
69	void	actualizarFormulario	private	—	Mostrar los campos necesarios cuando cambia la operación seleccionada.
70	String	leerDatoOperacion	public	String nombreCampo	Obtener el contenido de un campo del formulario de operación.

71	void	limpiarTabla	public	—	Quitar las filas mostradas antes de actualizar la tabla.
72	void	agregarFila	public	IntegracionAPI integracion	Mostrar los datos de una integración en una fila.
73	void	mostrarDetalles	public	IntegracionAPI integracion	Presentar los datos y la configuración de la integración.
74	void	mostrarResultado	public	String resultado	Presentar la respuesta de una operación y su auditoría.
75	void	mostrarMensaje	public	String mensaje	Mostrar avisos o errores de entrada mediante un cuadro de diálogo.
76	void	alBuscar	public	ActionListener accion	Registrar la acción del botón de búsqueda.
77	void	alListar	public	ActionListener accion	Registrar la acción del botón de listado.

78	void	alOrdenar	public	ActionListener accion	Registrar la acción del botón de ordenamiento.
79	void	alEjecutar	public	ActionListener accion	Registrar la acción del botón de ejecución.

Métodos para Main

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo
80	void	main	public static	String[] args	Crear el catálogo, cargar las integraciones iniciales, crear el motor, la vista y el controlador, e iniciar la aplicación.

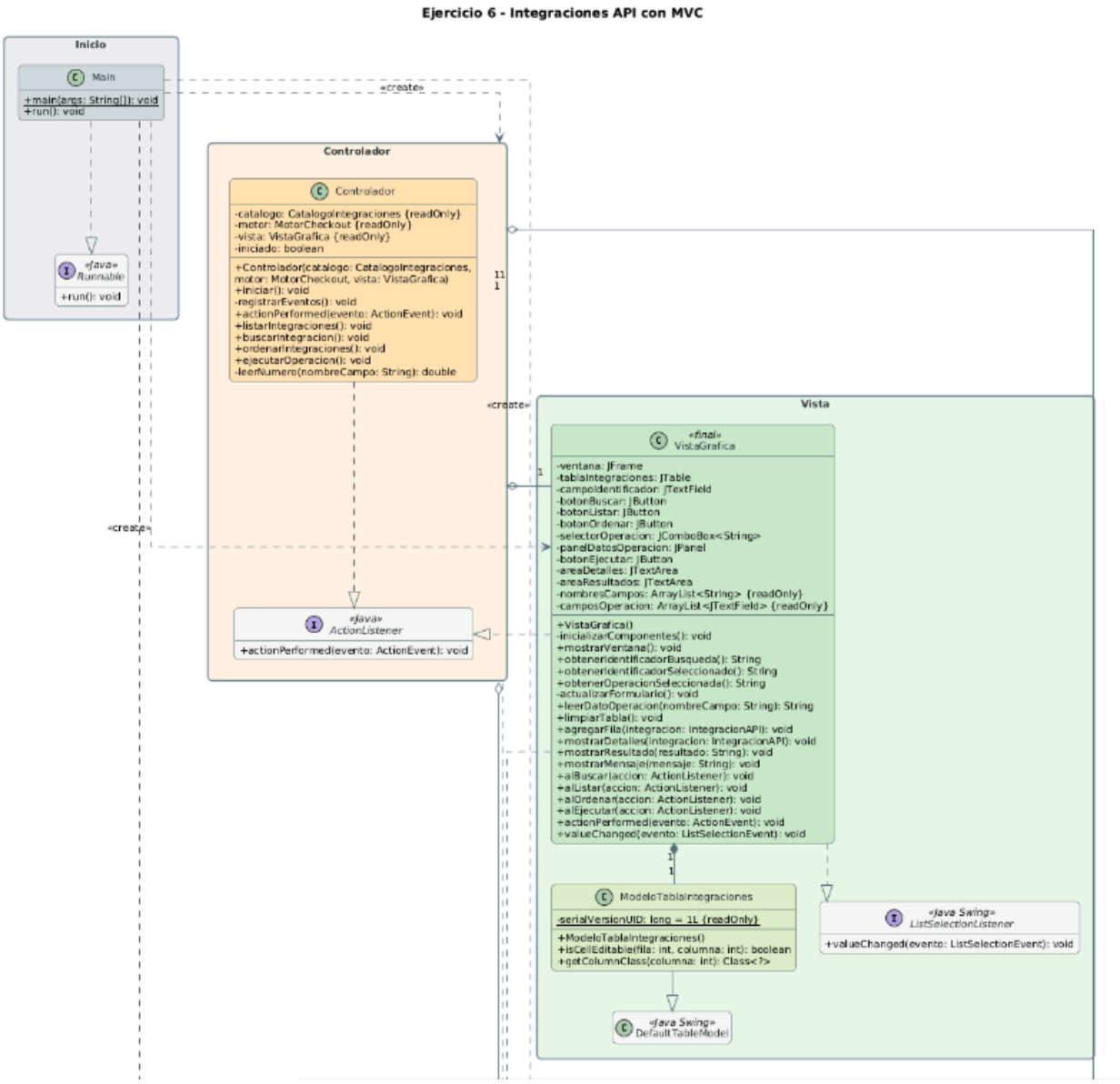
Métodos agregados posteriormente

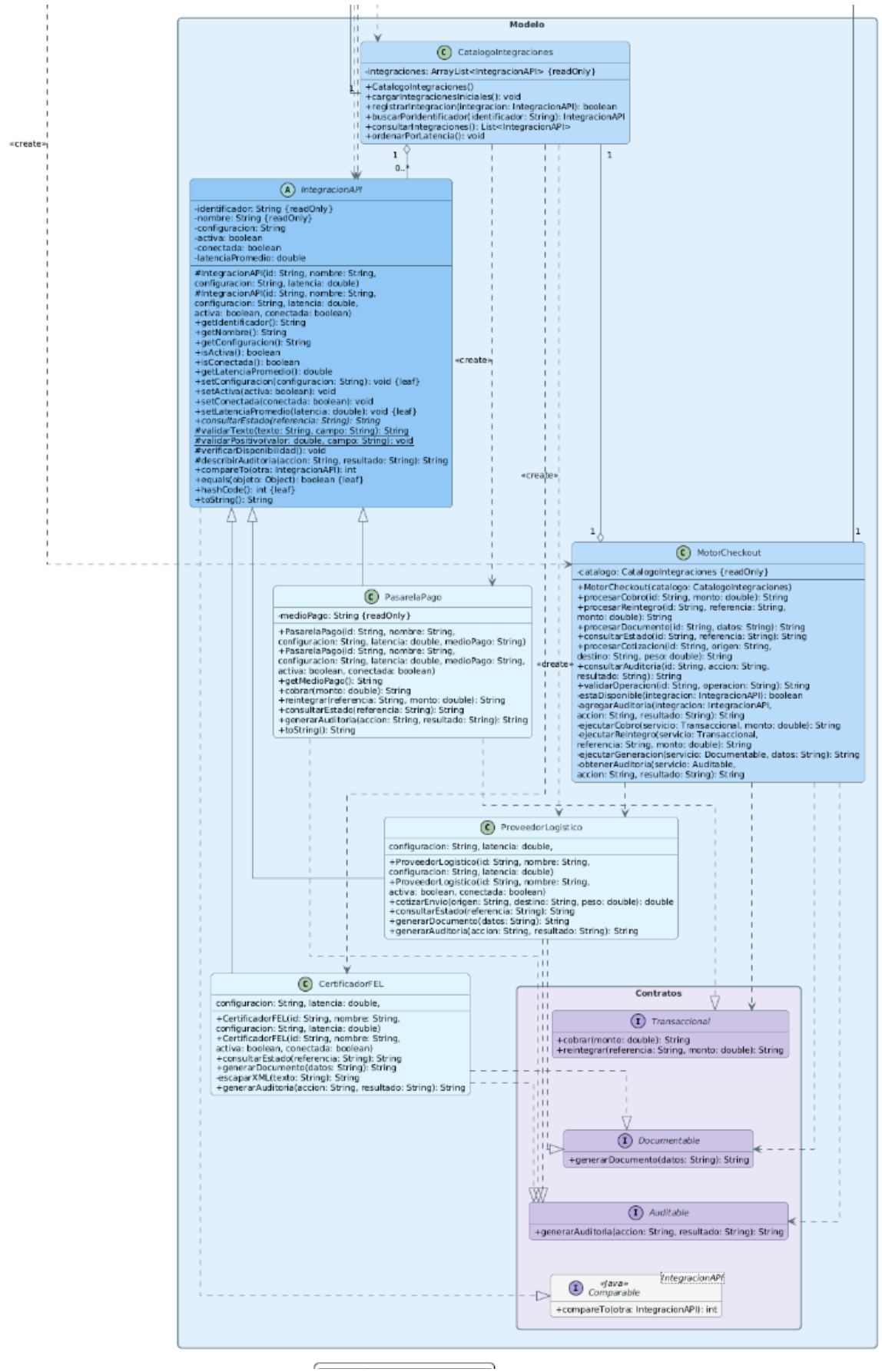
Clase	Retorno	Método y parámetros	Visibilidad	Motivo
IntegracionAPI	String	validarTexto(String texto, String campo)	protected static	Validar que un texto no esté vacío y quitar espacios de los extremos.
IntegracionAPI	void	validarPositivo(double valor, String campo)	protected static	Validar montos y pesos positivos y finitos.
IntegracionAPI	void	verificarDisponibilidad()	protected	Comprobar que el servicio esté activo y conectado.

IntegracionAPI	String	describirAuditoria(String accion, String resultado)	protected	Construir la descripción común de auditoría.
IntegracionAPI	boolean	equals(Object objeto)	public final	Comparar integraciones por su identificador.
IntegracionAPI	int	hashCode()	public final	Generar un código coherente con la comparación por identificador.
CertificadorFEL	String	escaparXML(String texto)	private	Adaptar caracteres especiales antes de incluirlos en el XML.
MotorCheckout	String	validarOperacion(String identificador, String operacion)	public	Comprobar existencia, capacidad y disponibilidad antes de continuar.
MotorCheckout	String	consultarAuditoria(String identificador, String accion, String resultado)	public	Generar una descripción de auditoría con los datos proporcionados.
MotorCheckout	String	agregarAuditoria(IntegracionAPI integracion, String accion, String resultado)	private	Adjuntar auditoría al resultado si el servicio implementa Auditable.

Controlador	void	actionPerformed(ActionEvent evento)	public	Identificar el botón pulsado y ejecutar su acción.
Controlador	double	leerNumero(String nombreCampo)	private	Convertir la entrada de texto en un número.
VistaGrafica	void	actionPerformed(ActionEvent evento)	public	Actualizar los campos al cambiar la operación seleccionada.
VistaGrafica	void	valueChanged(ListSelectionEvent evento)	public	Copiar el identificador seleccionado al campo de búsqueda.
ModeloTablaIntegraciones	—	ModeloTablaIntegraciones()	public	Crear las columnas de la tabla.
ModeloTablaIntegraciones	boolean	isCellEditable(int fila, int columna)	public	Impedir la edición de las celdas.
ModeloTablaIntegraciones	Class<?>	getColumnClass(int columna)	public	Indicar el tipo de dato de cada columna.
Main	void	run()	public	Crear y conectar los componentes desde el hilo de eventos de Swing.

UML


Ejercicio 6 - Integraciones API con MVC
Inicio
Main
+main(args: String[]): void
+run(): void
«create»
«java»
Runnable
+run(): void
Controlador
Controlador
-catalogo: CatalogoIntegraciones {readOnly}
-motor: MotorCheckout {readOnly}
-vista: VistaGrafica {readOnly}
-iniciado: boolean
+Controlador(catalogo: CatalogoIntegraciones, motor: MotorCheckout, vista: VistaGrafica)
+iniciar(): void
-registrarEventos(): void
+actionPerformed(evento: ActionEvent): void
+listarIntegraciones(): void
+buscarIntegracion(): void
+ordenarIntegraciones(): void
+ejecutarOperacion(): void
-leerNumero(nombreCampo: String): double
11
1
«create»
«create»
«java»
ActionListener
+actionPerformed(evento: ActionEvent): void
Vista
«final»
VistaGrafica
1
-ventana: JFrame
-tablaIntegraciones: JTable
-campoIdentificador: JTextField
-botonBuscar: JButton
-botonListar: JButton
-botonOrdenar: JButton
-selectorOperacion: JComboBox<String>
-panelDatosOperacion: JPanel
-botonEjecutar: JButton
-areaDetalles: JTextArea
-areaResultados: JTextArea
-nombresCampos: ArrayList<String> {readOnly}
-camposOperacion: ArrayList<JTextField> {readOnly}
+VistaGrafica()
-inicializarComponentes(): void
+mostrarVentana(): void
+obtenerIdentificadorBusqueda(): String
+obtenerIdentificadorSeleccionado(): String
+obtenerOperacionSeleccionada(): String
-actualizarFormulario(): void
+leerDatoOperacion(nombreCampo: String): String
+limpiarTabla(): void
+agregarFila(integracion: IntegracionAPI): void
+mostrarDetalles(integracion: IntegracionAPI): void
+mostrarResultado(resultado: String): void
+mostrarMensaje(mensaje: String): void
+alBuscar(accion: ActionListener): void
+alListar(accion: ActionListener): void
+alOrdenar(accion: ActionListener): void
+alEjecutar(accion: ActionListener): void
+actionPerformed(evento: ActionEvent): void
+valueChanged(evento: ListSelectionEvent): void
1
1
ModeloTablaIntegraciones
-serialVersionUID: long = 1L {readOnly}
+ModeloTablaIntegraciones()
+isCellEditable(fila: int, columna: int): boolean
+getColumnClass(columna: int): Class<?>
«Java Swing»
ListSelectionListener
+valueChanged(evento: ListSelectionEvent): void
«Java Swing»
DefaultTableModel


Modelo
CatalogoIntegraciones
-integraciones: ArrayList<IntegracionAPI> {readOnly}
+CatalogoIntegraciones()
+cargarIntegracionesIniciales(): void
+registrarIntegracion(integracion: IntegracionAPI): boolean
+buscarPorIdentificador(identificador: String): IntegracionAPI
+consultarIntegraciones(): List<IntegracionAPI>
+ordenarPorLatencia(): void
IntegracionAPI
-identificador: String {readOnly}
-nombre: String {readOnly}
-configuracion: String
-activa: boolean
-conectada: boolean
-latenciaPromedio: double
#IntegracionAPI(id: String, nombre: String, configuracion: String, latencia: double)
#IntegracionAPI(id: String, nombre: String, configuracion: String, latencia: double, activa: boolean, conectada: boolean)
+getIdentificador(): String
+getNombre(): String
+getConfiguracion(): String
+isActiva(): boolean
+isConectada(): boolean
+getLatenciaPromedio(): double
+setConfiguracion(configuracion: String): void {leaf}
+setActiva(activa: boolean): void
+setConectada(conectada: boolean): void
+setLatenciaPromedio(latencia: double): void {leaf}
+consultarEstado(referencia: String): String
#validarTexto(texto: String, campo: String): String
#validarPositivo(valor: double, campo: String): void
#verificarDisponibilidad(): void
#describirAuditoria(accion: String, resultado: String): String
+compareTo(otra: IntegracionAPI): int
+equals(objeto: Object): boolean {leaf}
+hashCode(): int {leaf}
+toString(): String
MotorCheckout
-catalogo: CatalogoIntegraciones {readOnly}
+MotorCheckout(catalogo: CatalogoIntegraciones)
+procesarCobro(id: String, monto: double): String
+procesarReintegro(id: String, referencia: String, monto: double): String
+procesarDocumento(id: String, datos: String): String
+consultarEstado(id: String, referencia: String): String
+procesarCotizacion(id: String, origen: String, destino: String, peso: double): String
+consultarAuditoria(id: String, accion: String, resultado: String): String
+validarOperacion(id: String, operacion: String): String
-estaDisponible(integracion: IntegracionAPI): boolean
-agregarAuditoria(integracion: IntegracionAPI, accion: String, resultado: String): String
-ejecutarCobro(servicio: Transaccional, monto: double): String
-ejecutarReintegro(servicio: Transaccional, referencia: String, monto: double): String
-ejecutarGeneracion(servicio: Documentable, datos: String): String
-obtenerAuditoria(servicio: Auditable, accion: String, resultado: String): String
PasarelaPago
-medioPago: String {readOnly}
+PasarelaPago(id: String, nombre: String, configuracion: String, latencia: double, medioPago: String)
+PasarelaPago(id: String, nombre: String, configuracion: String, latencia: double, medioPago: String, activa: boolean, conectada: boolean)
+getMedioPago(): String
+cobrar(monto: double): String
+reintegrar(referencia: String, monto: double): String
+consultarEstado(referencia: String): String
+generarAuditoria(accion: String, resultado: String): String
+toString(): String
ProveedorLogistico
configuracion: String, latencia: double,
+ProveedorLogistico(id: String, nombre: String, configuracion: String, latencia: double)
+ProveedorLogistico(id: String, nombre: String, activa: boolean, conectada: boolean)
+cotizarEnvio(origen: String, destino: String, peso: double): double
+consultarEstado(referencia: String): String
+generarDocumento(datos: String): String
+generarAuditoria(accion: String, resultado: String): String
CertificadorFEL
configuracion: String, latencia: double,
+CertificadorFEL(id: String, nombre: String, configuracion: String, latencia: double)
+CertificadorFEL(id: String, nombre: String, activa: boolean, conectada: boolean)
+consultarEstado(referencia: String): String
+generarDocumento(datos: String): String
-escaparXML(texto: String): String
+generarAuditoria(accion: String, resultado: String): String
Contratos
Transaccional
+cobrar(monto: double): String
+reintegrar(referencia: String, monto: double): String
Documentable
+generarDocumento(datos: String): String
Auditable
+generarAuditoria(accion: String, resultado: String): String
«java» Comparable
IntegracionAPI
+compareTo(otra: IntegracionAPI): int
«create»
«create»
«create»
«create»
«create»
1
0..*
1
1
1
1
1
1